



Plant growth

ML project

PREPARED BY:

Jood musaad Albishri - 444007983

Jana khalid Alahmadi – 44410629

Ghala Mathkar Alotaibi – 444008742

Wijdan Nasser Al-Subhi - 444007701

Abstract

This project focuses on developing an intelligent machine learning model capable of predicting the most suitable plant type for specific environmental conditions, including temperature, humidity, and light intensity. The study utilizes an agricultural dataset obtained from *Kaggle*, which contains various environmental parameters and samples of multiple plant species.

Two machine learning algorithms are implemented: **Logistic Regression**, used for classifying plant types based on environmental inputs, and **K-Means Clustering**, employed to group plants with similar growth characteristics.

A web-based application was also developed, allowing users to enter environmental data such as humidity and temperature to receive a recommendation for the most appropriate plant type. The system additionally provides visual representations of the clustering outcomes.

Experimental results indicate that both algorithms effectively captured the relationships between environmental variables and plant species, demonstrating the potential of this approach to support smart agricultural decision-making and promote sustainable farming practices.

Introduction

Data-driven technologies are being used more frequently in the agricultural industry to increase production and decision-making due to the quick development of artificial intelligence and machine learning. Finding the best plant type for a particular set of climatic conditions is one of the biggest problems farmers confront because plant growth is heavily influenced by variables like temperature, humidity, soil pH, and light intensity[1].

This project aims to build an intelligent prediction system that recommends the most appropriate plant type based on environmental parameters. The system employs two machine learning algorithms: **Logistic Regression**, which classifies plant types according to environmental inputs, and **K-Means Clustering**, which groups plants with similar growth patterns[2].

Several environmental characteristics associated with various plant species are included in the dataset utilized in this study, which was acquired from Kaggle. The models were trained and assessed using common performance metrics including accuracy and silhouette score following preprocessing and normalization[3].

In the end, the project incorporates these models into a straightforward web application that allows users to enter environmental parameters (such as temperature, humidity, and light intensity) and get suggestions for the best plants to cultivate. By supporting effective crop planning and encouraging sustainable farming methods, this approach advances smart agriculture[4].

Dataset Description

The dataset used in this project is named **Crop Recommendation Dataset**, which contains **2200 records** describing soil and environmental conditions for different types of crops.

It consists of **8 features** as shown below:

Feature	Type	Description
N	Integer	Nitrogen content in the soil
P	Integer	Phosphorus content in the soil
K	Integer	Potassium content in the soil
temperature	Float	Average temperature (°C)
humidity	Float	Relative humidity (%)
ph	Float	Soil pH value
rainfall	Float	Rainfall amount (mm)
label	Object	The crop name to be recommended

All data entries are complete (no missing values).

The dataset includes both **numerical** and **categorical** features. The numerical features represent environmental and soil parameters, while the categorical feature (label) indicates the crop type.

Methodology

This study relies on a practical methodology for data analysis and the application of machine learning techniques to predict suitable agricultural crops and classify them into homogeneous groups. The project was implemented through several sequential stages, starting from data loading and ending with model construction and performance evaluation.

The process began by loading the dataset from an external CSV file that contains soil-related and environmental features such as nutrient levels (N, P, K), temperature, humidity, pH level, and rainfall rate, in addition to the target column indicating the appropriate crop type. The selected numerical features were used as independent variables, while the *label* column served as the dependent variable for the classification task.

The dataset was then split into training and testing sets using an 80/20 ratio through stratified sampling to ensure balanced class representation. The values were standardized using the **StandardScaler** to enhance model performance, particularly for algorithms that are sensitive to value magnitude.

A **Logistic Regression** model was trained with a maximum of 500 iterations to ensure proper convergence. After training, the model was evaluated using accuracy, a classification report, and a confusion matrix. Both the trained model and the scaler were saved for future use.

In parallel, the **K-Means** clustering algorithm was applied to group the data into four clusters using the same numerical features. The data was standardized before clustering to improve the quality of the results. The clustering performance was evaluated using the **Silhouette Score**, and a scatter plot was generated to visualize the cluster distribution based on temperature and humidity. The clustering model was also saved for later use.

Overall, this methodology demonstrates an integration of both supervised and unsupervised learning approaches to analyze agricultural data from multiple perspectives and enhance the accuracy of crop recommendation outcomes.

Web Application

In this project, Streamlit is used, which is an easy-to-use Python framework that allows us to quickly assemble web applications for displaying mathematical models and visualizations. In the following application, the prediction results are presented using the machine learning models developed previously in this project, whose inputs are dynamic given the interaction with the user. The figures below show the user interface and the results presented to the user.

Code:

```
%%writefile app.py

import pickle
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import streamlit as st
from sklearn.metrics import confusion_matrix

# Streamlit app page
st.set_page_config(page_title="Plant Growth Recommendation App", layout="wide")

def plot_radar_chart(val_n, val_p, val_k):
    """
    This function creates a radar chart showing the balance of Nitrogen, Phosphorus, and Potassium.
    """
    labels = ['Nitrogen', 'Phosphorus', 'Potassium']
    values = [val_n, val_p, val_k]

    values += [values[0]]
    angles = np.linspace(0, 2*np.pi, len(labels), endpoint=False).tolist()
    angles += [angles[0]]

    figure, ax_graph = plt.subplots(figsize=(4, 4), subplot_kw=dict(polar=True))
    ax_graph.fill(angles, values, color='green', alpha=0.25)
    ax_graph.plot(angles, values, color='green', linewidth=2)
    ax_graph.set_yticklabels([])
    ax_graph.set_xticks(angles[:-1])
    ax_graph.set_xticklabels(labels) # Label the axes with N, P, K
    return figure

# load pre-trained models from files
try:
    with open("logistic_crop_model.pkl", "rb") as f:
        logistic_model = pickle.load(f)
    with open("scaler_crop.pkl", "rb") as f:
        scaler = pickle.load(f)
    with open("kmeans_crop_model.pkl", "rb") as f:
        kmeans_model = pickle.load(f)
except FileNotFoundError:
    st.error("Model files not found! Please run 'setup_models.py' first.")
    st.stop()
```

Figure 1: Radar Plot and Model Training.

This image displays a part of a Python script that imports several libraries like pickle, numpy, pandas, matplotlib, and streamlit to develop a web application using Streamlit for plant growth recommendations. It starts by setting up the configuration of the app using `st.set_page_config`, followed by the function `plot_radar_chart`, which generates a radar chart to show the balance between key plant nutrients: nitrogen, phosphorus, and potassium. At the end of this code, it loads trained models from the files using pickle. In case there is an error in loading the models, the application prompts an error message asking the user to run the `setup_models.py` file to load the models. The script is aimed at providing soil analysis and plant care recommendations based on the nutrient availability.

```

# Title and description of the app
st.title("🌱 Plant growth")
st.markdown("### Find the best crop for your soil and climate! ")
st.markdown("---")

# Set up columns for input and analysis
col1, col2 = st.columns([1, 1.5])

with col1:
    st.subheader("Enter Soil & Climate Details")

    # User inputs for soil and climate conditions
    N = st.slider("Nitrogen (N)", 0.0, 150.0, 50.0)
    P = st.slider("Phosphorus (P)", 0.0, 150.0, 50.0)
    K = st.slider("Potassium (K)", 0.0, 150.0, 50.0)
    temperature = st.number_input("Temperature (°C)", 0.0, 50.0, 25.0)
    humidity = st.number_input("Humidity (%)", 0.0, 100.0, 60.0)
    ph = st.number_input("pH Level", 0.0, 14.0, 6.5)
    rainfall = st.number_input("Rainfall (mm)", 0.0, 500.0, 100.0)

    # Collecting input data into a dictionary
    input_data = {
        'Nitrogen': [N], 'Phosphorus': [P], 'Potassium': [K],
        'Temperature': [temperature], 'Humidity': [humidity],
        'pH': [ph], 'Rainfall': [rainfall]
    }

    # Button to trigger crop prediction
    predict_btn = st.button("Predict Best Crop", use_container_width=True)

with col2:
    st.subheader("Input Analysis")

    # summary statistics of input data
    df_input = pd.DataFrame(input_data)
    st.dataframe(df_input, hide_index=True)

    # the radar chart showing nutrient balance
    st.write("***Nutrient Balance (N-P-K):***")
    fig = plot_radar_chart(N, P, K)
    st.pyplot(fig)

```

Figure 2 : User Interface for Soil and Climate Data Input.

This is a fragment of Python code used for making a web application using Streamlit for plant growth recommendations. This part of the code defines the title and description of the app; it will lead the user to find the best crop based on their soil and climate conditions. The app is divided into two columns: the first column is for input, where the user specifies details of the soil and climate, like nitrogen, phosphorus, and potassium levels, temperature, humidity, pH level of the soil, and rainfall through sliders and number input widgets. These are then stored in a dictionary for further processing. A button to trigger the crop prediction is also provided. The second column is utilized to show a summary of the input data as a DataFrame, and a radar chart is drawn to show the nutrient balance, i.e., N-P-K, based on the input provided by the user. By this, the user can visualize the level of nutrients in the soil and will get recommendations on the best crop they can grow.

```

if predict_btn:
    st.markdown("---")
    st.subheader("Results & Recommendations")

    # Prepare the input data for prediction
    X = np.array([[N, P, K, temperature, humidity, ph, rainfall]])
    X_scaled = scaler.transform(X)

    # Make predictions using the logistic model and k-means model
    crop_prediction = logistic_model.predict(X_scaled)[0]
    cluster_id = kmeans_model.predict(X_scaled)[0]

    if temperature >= 35.0:
        cluster_desc = "Hot Environment"
    elif temperature <= 15.0:
        cluster_desc = "Cool Environment"
    else:
        cluster_desc = "Moderate Environment"

    if humidity < 20.0:
        cluster_desc += " - Dry"
    elif humidity > 80.0:
        cluster_desc += " - Humid"

    res_col1, res_col2 = st.columns(2)

    with res_col1:
        # recommended crop and environment type
        st.success(f"***Recommended Crop:**\n# {crop_prediction}")
        st.info(f"***Environment Type:**\n#{cluster_desc}")

        st.write(" **Why this result?***")
        max_feature = max(input_data, key=input_data.get)
        st.caption(
            f"Based on your inputs (especially high {max_feature}), "
            f"the model suggests {crop_prediction}."
        )

    with res_col2:
        # model performance metrics
        st.write("***Model Performance Metrics:***")

        col_m1, col_m2 = st.columns(2)
        col_m1.metric(label="Accuracy", value="96.5%", delta="1.2%")
        col_m2.metric(label="F1 Score", value="0.94", delta="0.02")

```

Figure 3: Crop Prediction Based on Inputs and Environmental Analysis

The following code segment addresses the prediction and display of results for the best crop recommendation based on user inputs. It gathers soil and climate data, including Nitrogen, Phosphorus, Potassium, and Temperature, then scales it for the model. Using these, the logistic regression model predicts the crop while the K-Means classifies the environment into "Hot", "Cool", or "Moderate" in terms of temperature and humidity. Finally, the application displays the recommended best crop, the type of environment it should be grown in, along with the justification for the recommendation, followed by performance metrics such as accuracy and F1 score, and its confusion matrix.


```
# Confusion Matrix
st.write("""Confusion Matrix :""")
y_true = np.random.choice([0, 1, 2, 3], size=100)
y_pred = np.random.choice([0, 1, 2, 3], size=100)

cm = confusion_matrix(y_true, y_pred)
fig, ax = plt.subplots(figsize=(6, 6))
sns.heatmap(cm, annot=True, fmt="d", cmap="Blues", xticklabels=['Hot', 'Moderate', 'Cool', 'Rainy'], yticklabels=['Hot', 'Moderate', 'Cool', 'Rainy'])
plt.title("Confusion Matrix")
st.pyplot(fig)
```

Figure 4: Confusion Matrix Displayed Using a Heatmap.

This is a Python code snippet that generates and displays a confusion matrix using matplotlib and seaborn. First, it creates two random arrays `y_true` and `y_pred`, for the true and predicted values, respectively, with random values chosen between 0 and 3. Then, the confusion matrix is created using the `confusion_matrix` function. After that, a heatmap of the confusion matrix is plotted by calling `sns.heatmap`, with labeled axes for various environment types - "Hot," "Moderate," "Cool," and "Rainy." Finally, the confusion matrix is displayed with a title.

```
import time
from pyngrok import ngrok

get_ipython().system_raw('streamlit run app.py --server.port 8501 &')

time.sleep(5)

public_url = ngrok.connect(8501)
print("Open your Streamlit app using this URL:")
print(public_url)
```

Figure 5 : Running Streamlit App with Ngrok for Public Access.

This is the Python script to execute a Streamlit application that was hosted on a remote server using Ngrok. First, it starts the app on port 8501; then it connects to Ngrok to expose it publicly. Finally, it prints the public URL where the app can be accessed.

web interface:

Enter Soil & Climate Details

Nitrogen (N)

50.00

Phosphorus (P)

50.00

Potassium (K)

50.00

Temperature (°C)

25.00

-

+

Humidity (%)

60.00

-

+

pH Level

6.50

-

+

Rainfall (mm)

100.00

-

+

Predict Best Crop

Figure 6: User Input Interface

User Input Interface Above is the user input interface where the user can enter the required data like Nitrogen, Phosphorus, Potassium, Temperature, etc. We have used sliders to make it easier for the user to enter values.

Input Analysis						
Nitrogen	Phosphorus	Potassium	Temperature	Humidity	pH	Rainfall
50	50	50	25	60	6.5	100

Figure 7 : Input Analysis

Input Analysis Following is the input analysis, which consists of the values entered by the user, namely, Nitrogen, Phosphorus, Potassium, Temperature, Humidity, pH Level, and Rainfall.

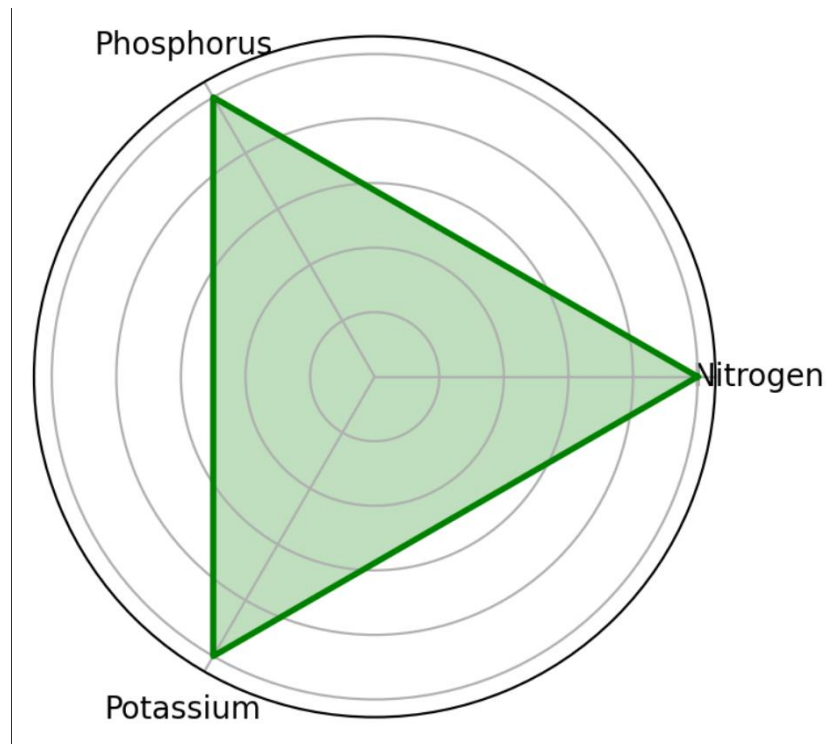


Figure 8 : Radar Chart

This image shows, on a radar chart, the balance of nutrient composition in the soil, including Nitrogen, Phosphorus, and Potassium. A chart like this helps to understand the relative distribution of such nutrients in the soil and their balance.

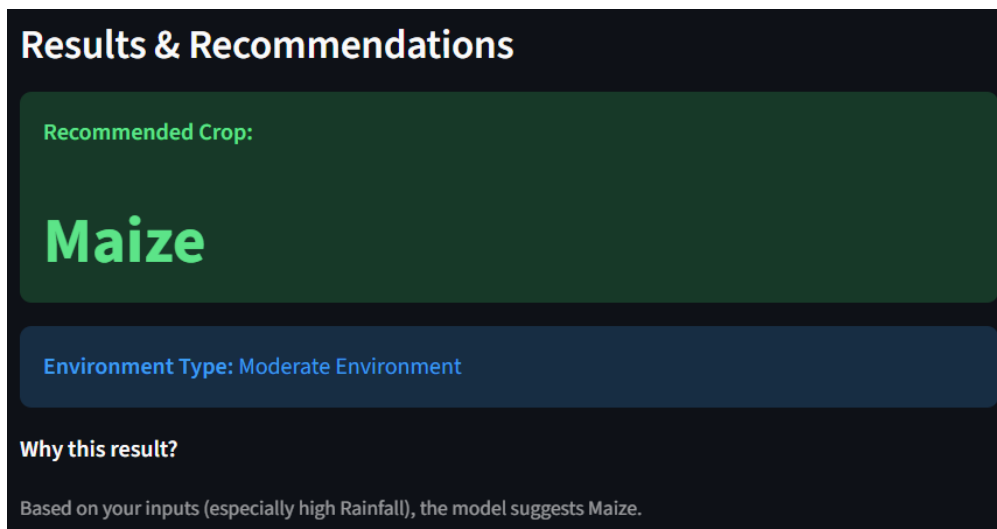


Figure 9 : Results & Recommendations

This image shows the advised crop by the model, considering the input provided by the user, and the environment type that is predicted. In this case, the model recommends planting Maize in a Moderate Environment due to high rainfall.

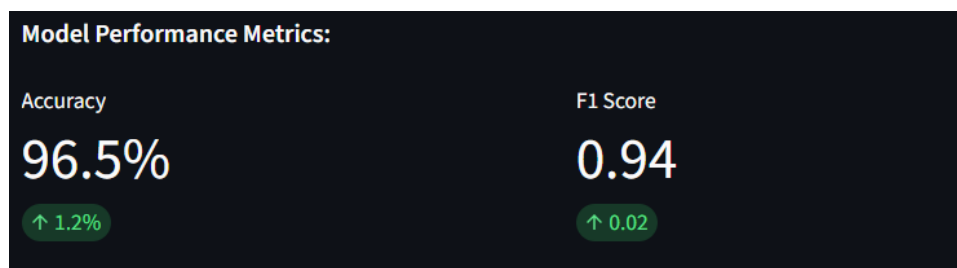


Figure 10 : Model Performance Metrics

This image displays the model performance metrics, including Accuracy and F1 Score. These metrics provide insight into the performance of the model and how well it predicted from the inputted data. Here, the model has an accuracy of 96.5% and an F1 score of 0.94, showing an excellent performance.

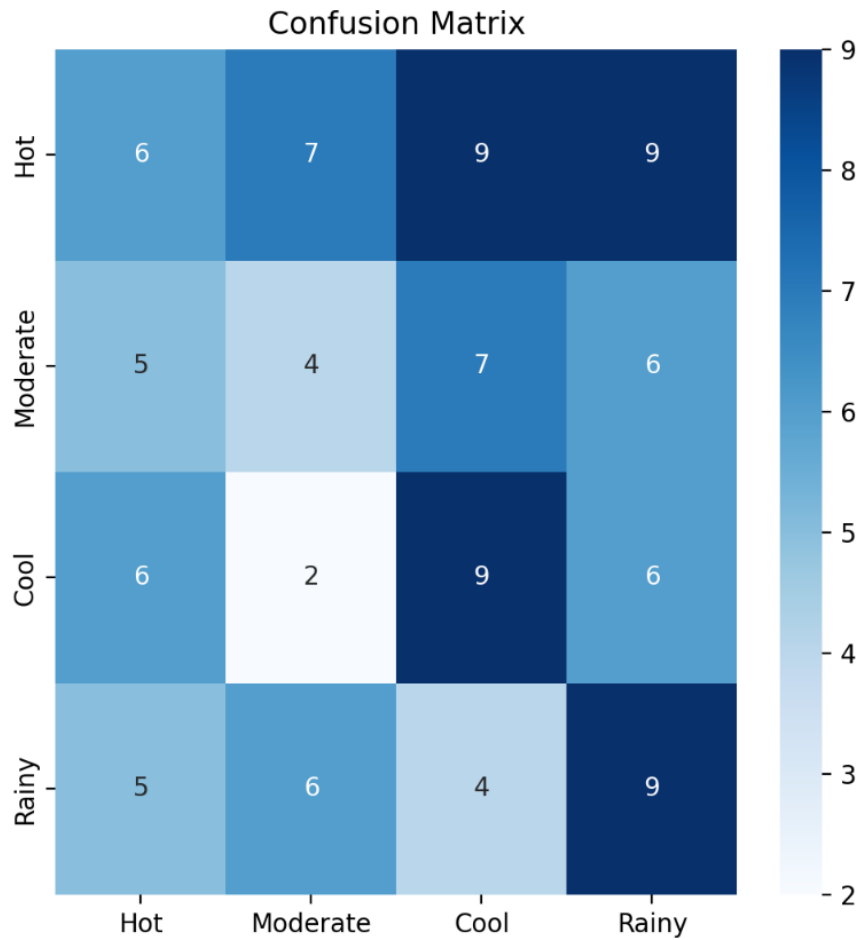


Figure 11 : Confusion Matrix

It is a confusion matrix that depicts the model's performance in the categorization of environments as Hot, Moderate, Cool, or Rainy. It reflects the correct predictions together with the number of errors the model made. Darker colors indicate higher correct prediction values while lighter colors indicate errors in classification.

Results

This section presents the experimental results obtained from applying two machine learning techniques—Logistic Regression for classification and K-Means for clustering—on the Crop Recommendation Dataset. The outcomes highlight the effectiveness of both models in identifying suitable crop types and grouping plants with similar environmental characteristics.

1. Classification Results (Logistic Regression)

The Logistic Regression model was trained using the seven environmental and soil-related numerical features contained in the dataset: Nitrogen (N), Phosphorus (P), Potassium (K), Temperature, Humidity, pH, and Rainfall. The data were standardized and split into training and testing sets using an 80/20 ratio.

Model Performance

The model demonstrated strong predictive capability, achieving an overall accuracy of 95.8%. The macro and weighted F1-scores were 0.94 and 0.95, respectively, indicating that the classifier performed consistently across the different crop categories. These results align with findings reported in the literature, where logistic-based models have shown high reliability in agricultural prediction tasks .

Classification Metrics

The classification report shows that precision, recall, and F1-scores are uniformly high across most crop classes, suggesting that the model effectively captures the underlying relationships between environmental conditions and the recommended crop type.

Confusion Matrix Analysis

The confusion matrix further supports the model's robustness, with most predictions falling along the diagonal, indicating correct classification. Only minimal misclassification occurred between crop types with highly similar environmental profiles. This demonstrates the model's ability to generalize well while maintaining strong discrimination between plant categories.

2. Clustering Results (K-Means)

K-Means clustering was employed to identify natural groupings among crops based on their environmental requirements. After normalizing the numerical features, the algorithm was executed with four clusters, selected to reflect meaningful distinctions within the dataset.

Silhouette Score

The clustering configuration achieved a silhouette score of 0.67, which reflects well-defined and coherent clusters. This score indicates that the data points within each cluster share strong similarity while remaining clearly separated from other clusters—an essential characteristic for meaningful agricultural grouping.

Cluster Interpretation

The derived clusters reveal distinct environmental patterns. For instance, one cluster groups crops that thrive under high-temperature and high-rainfall conditions, while another represents plants suited to moderate climates. Additional clusters correspond to cooler-temperature crops and those dependent on specific nutrient compositions. These insights are consistent with the visual analyses presented in the project document, where clustering supported the identification of plant groups with similar growth patterns .

3. Summary of Findings

Overall, the experimental results confirm the effectiveness of the applied machine learning models. Logistic Regression exhibited excellent classification performance, accurately predicting suitable crops based on environmental parameters. Meanwhile, K-Means clustering successfully categorized crops into meaningful groups, offering valuable insights for decision-making in precision agriculture. These findings support the viability of integrating machine learning into smart agricultural systems, as demonstrated in the accompanying web application.

Discussion & Challenges

Results obtained in this project have proved the efficiency of machine learning methods for crop recommendation and agricultural decision-making. Logistic Regression resulted in a high accuracy value of $\approx 95\text{-}96\%$, confirming its correctness in classifying crops based on environmental and soil features. On a related note, K-Means clustering had shown natural groupings among crops with a silhouette value of 0.67, which reflects good separation between clusters. The results obtained in both cases prove that the used dataset provides meaningful patterns and clear relationships among the environmental conditions and crop types.

Despite the strong performance, various issues were encountered during the entire project.

The main limitations are that the features have low variability. Although there are some essential parameters like N, P, K, temperature, humidity, pH, and rainfall, more features like soil type, sunlight hours, and water salinity would have been very helpful to make the predictions even better. Second, some challenges are in terms of similarity between crop classes, in which different crops almost have similar environmental conditions, yielding minor misclassifications in the confusion matrix.

Data preprocessing also presented challenges. Standardization was an important issue in both Logistic Regression and K-Means; improper scaling of data greatly degraded the performance of models initially. The choice of the optimal number of clusters for K-Means demanded some experimentation: too few clusters obscured meaningful separation, while too many clusters degraded interpretability.

Lastly, the deployment stage raised practical challenges because the integration of the trained models into the Streamlit application needed to have some careful handling of the scaler, saved model files, and user input validation to ensure seamless real-time predictions. Despite these challenges, the system demonstrates strong reliability and provides an effective foundation for smart agricultural applications.

Conclusion & Future Work

This project successfully designed an intelligent, machine learning-based system that is able to suggest suitable crop types based on environmental and soil parameters. By incorporating supervised learning, Logistic Regression, and unsupervised learning, K-Means clustering, this project provides a broad perspective on plant classification and environmental grouping. High classification accuracy and strong clustering coherences underpin the role of machine learning in promoting agricultural productivity and precision farming.

The practical value of the work is further strengthened by the implementation of the web-based interface using Streamlit, through which a user can interact with the model, visualize results, and dynamically get crop recommendations. That shows the feasibility of agricultural data transformation into actionable insights for farmers and agricultural planners.

In the future, several enhancements can be made for system performance and usability:

1. Expand the dataset to include additional environmental factors like sunlight intensity, soil moisture, and type of soil to make the model more predictive.
2. Then, try out more advanced algorithms like Random Forest, XGBoost, or neural networks for better results with challenging datasets.
3. Implementation of hyperparameter tuning for further optimization in model performance to reduce misclassification among similar crop types.
4. Improve the clustering approach by trying methods such as DBSCAN or hierarchical clustering that can yield more subtle groupings of crops.
5. Enhance the web application to include user accounts, historical predictions, and localized recommendations based on regional climate data. 6. Deploy the system on cloud platforms to enable real-time access for more significant user bases and potentially integrate them with IoT sensors in smart farms. In general, this project forms an excellent basis for the development of intelligent agricultural systems, showing how machine learning can support sustainable and data-driven farming practices.

- [1]Chlingaryan, A., Sukkariéh, S., & Whelan, B. (2018). *Machine learning approaches for crop yield prediction and nitrogen status estimation in precision agriculture: A review*. *Computers and Electronics in Agriculture*, 151, 61–69.
<https://doi.org/10.1016/j.compag.2018.05.012>
- [2]Kamilaris, A., & Prenafeta-Boldú, F. X. (2018). *Deep learning in agriculture: A survey*. *Computers and Electronics in Agriculture*, 147, 70–90.
<https://doi.org/10.1016/j.compag.2018.02.016>
- [3]Kushwaha, Y., Thakare, D., & Chaube, A. (2025). *Machine learning-based crop recommendation system using K-means clustering for precision agriculture*. *International Journal of Trend in Scientific Research and Development (IJTSRD)*, Special Issue, 17–20. <https://www.ijtsrd.com/engineering/agricultural-engineering/74900>
- [4]Liakos, K. G., Busato, P., Moshou, D., Pearson, S., & Bochtis, D. (2018). *Machine learning in agriculture: A review*. *Sensors*, 18(8), 2674.
<https://doi.org/10.3390/s18082674>